RESEARCH PROJECT • ASSIGNMENT 3

Neural Prediction and Vector Drift

One team of two • From token counts to continuous representations • ACL-style paper v0.3

1. Why this assignment matters

Assignments 1 and 2 established a reproducible character-noise experiment and count-based language models. Assignment 3 keeps the same research question and corpus, but changes how context is represented: token IDs now select dense vectors, and a small neural network learns next-token probabilities.

You will investigate two linked questions: how much does noise change a static vector representation, and how does it change next-token prediction? These are different measurements. A small vector displacement is not proof that meaning has been preserved.

You will build two small neural networks that predict the next token from the preceding two tokens. The first averages those two token vectors (mean pooling); the second joins them in order (concatenation). Both use the same fixed pretrained token vectors and the same basic network structure.

These are different measurements. A small vector displacement is not proof that meaning has been preserved.

Workstream	Required investigation
Student A: geometry	Extract a frozen pretrained embedding table. Measure cosine similarity and Euclidean distance for aligned clean/noisy text spans.
Student B: prediction	Train two small feedforward next-token models: mean-pooled context versus ordered concatenation. Evaluate loss, perplexity and accuracy.
Both: research	Verify the code, compare with the Assignment 2 WordPiece baseline, interpret counterexamples and extend the same ACL-style paper.

2. Research claims to test, not assume

- Does character noise change WordPiece segmentation and the pooled vectors? Does the amount of change depend on noise type and rate?
- Does either neural model show a smaller performance deterioration than the matched n-gram baseline? Does preserving context order help?
- Do vector changes and prediction changes move together, or are there counterexamples?

Do not promise a neural advantage, monotonically falling cosine similarity, or a universal failure threshold at p > 0.20. Report null, mixed and contrary results honestly. This is a controlled study, not a demonstration with a predetermined answer.

Reading: Jurafsky & Martin, supplied third-edition draft, Chapters 5 and 6; the lecture introduces the architecture and pretraining ideas used in later work.

RESEARCH PROJECT • ASSIGNMENT 3

Keep the experiment continuous

The same corpus, paired records, tokenizer policy and corruption rules carry forward.

3. Data, splits and bounded sampling

Reuse the saved Wikipedia subcorpus, train/validation/test split membership, record IDs and clean/noisy manifests from Assignments 1–2. Train on clean training records only. Use validation for checkpoint selection and preliminary research results. Keep the test split unopened for the semester's final, frozen evaluation plan.

Use the same training and validation records as Assignment 2. If computational limits require a smaller sample, agree on the reduced size with the instructor before conducting the experiments. Select records reproducibly using their existing order, document the selection, and use the same retained records for both neural models and the matched n-gram baseline.

Save the chosen IDs, ordering, caps, actual counts and reason for any reduction. Filter the previously saved noisy records by these IDs; do not regenerate noise using a different sampling order. Use exactly the same retained records for both neural models and the comparison baseline. Do not truncate noisy text independently or replace difficult examples.

4. Required corruption grid

Setting	Required value
Noise operations	Insertion, deletion and substitution, exactly as implemented and documented in Assignment 2.
Noise rates p	0 (one shared clean condition), 0.01, 0.05, 0.10 and 0.20.
Corruption seeds	11, 23 and 47 for each nonzero rate and operation.
Eligible positions	Original ASCII letters. Preserve whitespace, punctuation, digits and record boundaries; retain the Assignment 2 edit rules and logs.

The requested rate p is not the measured edit distance. Preserve actual edit counts, realized minimum edit distance and normalization policy from the earlier pipeline. Save one clean baseline rather than pretending it is three independent seed runs.

RESEARCH PROJECT • ASSIGNMENT 3

Tokens become vectors

An embedding is a learned numeric representation, not a spelling-distance guarantee.

5. One fixed WordPiece vocabulary

Use the same fixed tokenizer checkpoint, revision, casing/accent policy and output support as Assignment 2. The default checkpoint is google-bert/bert-base-uncased. Record the exact tokenizer/checkpoint version and its vocabulary size. Assignment 3 requires only WordPiece; the whitespace results remain part of Assignment 2.

Keep the Assignment 2 allowed outputs: exclude padding, classification/separator/mask and unused placeholder entries; retain the unknown token and the project's end-of-record output.

6. Extract and freeze the lookup table

$$E \in \mathbb{R}^{|V_t| \times d}$$

E is the pretrained input embedding matrix. V_t is the tokenizer vocabulary indexing its rows; $|V_t|$ is its number of entries. d is the number of coordinates per vector (768 for the default checkpoint). Token ID j retrieves row *E*[j, :]. K, used later, is the number of allowed prediction outputs; it can differ from $|V_t|$.

Use only BERT's stored token vectors as the lookup table *E*. Keep these vectors unchanged in both neural models (mean-pooling and concatenation), and train only your own prediction-network weights and biases. Do not use BERT's other components or rescale the stored vectors. Check before and after training that *E* has not changed.

Term	Meaning in this assignment
Static	At a fixed model state, the same token ID receives the same lookup vector regardless of surrounding words.
Frozen	The optimizer does not update the embedding table.
Contextual	A representation changes with surrounding tokens after contextual processing. Full BERT representations belong to later work.

Pretrained embeddings contain information learned outside your selected corpus. Disclose the checkpoint's external pretraining and possible Wikipedia overlap. This is not a comparison in which all systems learned solely from the small class corpus.

RESEARCH PROJECT • ASSIGNMENT 3

Workstream A: measure vector drift

Write a Python program named noise_vector_drift.py to measure how character noise changes text-vector representations. Use cosine similarity and Euclidean distance. These measurements describe changes in the vectors; they do not directly measure changes in meaning.

7. Align text spans before comparing vectors

Before inspecting outcomes, select the first up to 1,000 nonempty whitespace-delimited clean spans containing an eligible letter from the retained validation records, in saved record order. Use the same span IDs in every condition. Preserve each original record ID, span index and character offsets.

Use the saved edit logs to recover each span's noisy counterpart, including inserted letters associated with its original positions. A fully deleted span must remain an explicitly empty paired span. Do not pair the nth clean subword with the nth noisy subword: noise can change the number of pieces. Do not lose empty spans by splitting and reindexing the corrupted text.

Tokenize each clean/noisy span separately with the fixed tokenizer, without added special tokens. Average its raw token-embedding rows to obtain one d-dimensional vector. This variable-length span pooling is a diagnostic; it is distinct from the fixed two-token model contexts in Section 11.

8. Two geometric measurements

$$cos(u, v) = \frac{u \cdot v}{\|u\|_2 \|v\|_2}; \quad D(u, v) = \|u - v\|_2$$

u is the clean span's mean vector; v is its noisy counterpart. The dot product u · v multiplies corresponding coordinates and sums them. $\|u\|_2$ is vector length. Cosine measures direction agreement (−1 to 1); D is Euclidean distance and also reflects vector magnitude. Higher cosine and smaller D indicate smaller geometric change—not necessarily preserved meaning.

Use raw pooled vectors for the required Euclidean result. Normalization is not required to make Euclidean distance valid. An optional unit-normalized analysis must be labeled separately; for unit vectors, squared distance equals 2(1 − cosine), so those two measurements are redundant.

For empty spans, or any zero-norm pooled vector, mark the pair undefined rather than assigning zero similarity or replacing it. Report selected, valid and undefined counts for every condition. Aggregate mean cosine and mean distance over valid pairs; report coverage alongside them. For p = 0, valid identical pairs should have cosine 1 and distance 0, within numerical tolerance.

RESEARCH PROJECT • ASSIGNMENT 3

Workstream B: predict the next token

Write a Python program named neural_next_token.py to build and train the two next-token prediction models: mean pooling and concatenation. Each model uses the preceding two tokens to predict the next token. Evaluate both models on the same clean and noisy text using loss, perplexity, and next-token accuracy.

9. What the network learns to predict

For each text record, use the two immediately preceding tokens as context to predict the next observed token. The text itself supplies the correct prediction targets, so no separately labeled dataset is needed. Start a fresh context at each record boundary. Do not use future tokens or carry context from one record into another.

Two-token context	Observed target
BOS, BOS	First token in the record
BOS, first token	Second token, if present
Previous two tokens	Next token in the same record
Final two context slots	One end-of-record token (EOS)

BOS means *beginning of sequence* and is input-only. Give each BOS slot a fixed all-zero vector of length d; do not add a new row to the pretrained checkpoint. EOS means *end of sequence* and is output-only here. A one-token record uses BOS and that token to predict EOS. An empty corrupted record uses BOS, BOS to predict EOS.

Tokenize complete retained records without BERT's added [CLS]/[SEP] wrappers or an automatic 512-token truncation. Each neural model uses only the preceding two tokens to predict the next token. Move this window through the entire retained record, scoring every observed token and one end-of-record token. Use the same set of allowed output tokens and the same end-of-record rules as Assignment 2.

10. Predicting each next token using the observed text

During training and evaluation, construct each context from the observed tokens in that condition. Clean evaluation uses clean prefixes and clean targets; noisy evaluation uses noisy prefixes and noisy targets. Do not feed sampled model outputs back into the evaluation sequence.

This measures likelihood of the observed text after tokenization. It does not directly measure recovery of the original clean words. A lower noisy perplexity can result from easier mapped targets or more unknown tokens, so always inspect tokenization diagnostics.

RESEARCH PROJECT • ASSIGNMENT 3

Compare mean pooling with concatenation

Keep the two-token context, frozen table, records and training settings fixed.

11. Two inputs, one network pattern

$$x_{mean} = (e_1 + e_2) / 2 ; \quad x_{concat} = [e_1; e_2]$$

e_1 and e_2 are the two context vectors in order, including any zero BOS vector. x_mean has d coordinates; x_concat has 2d coordinates. Always divide the mean by two. Concatenation preserves separate positions; the mean cannot distinguish a swap of the same two static vectors.

$$h = ReLU(W_1 x + b_1) ; \quad z = W_2 h + b_2$$

$$\hat{y}_k = \frac{exp(z_k)}{\Sigma_j \, exp(z_j)}$$

x is the chosen input vector; h is the hidden activation; ReLU(a) = max(0, a) coordinate by coordinate. z contains K unrestricted output scores (logits). $\hat{y}_k$ is the predicted probability of output class k. The sum runs over all K allowed outputs. W_1, b_1, W_2 and b_2 are learned weights and biases.

Parameter	Shape (column-vector convention)
W_1 and b_1	64 × m and 64; m = d for mean pooling or 2d for concatenation.
W_2 and b_2	K × 64 and K. Use a new output head, not BERT's prediction head.
Trainable count	64m + 64 + 64K + K. Exclude frozen E; report both trainable and frozen counts.

This is one hidden layer and two learned affine layers, excluding lookup. Concatenation has more input-layer parameters: record this limitation rather than attributing all differences solely to word order.

12. Training settings and model selection

Train the mean-pooling and concatenation models separately, using the same clean training records and the following settings:

Setting

Hidden-layer size: 64

Optimizer: Adam; learning rate: 0.001

Batch size: 128

Maximum: 5 epochs

Setting

Random seed: 101

No dropout or weight decay

Keep the pretrained token-embedding table unchanged. Train only the weights and biases of your prediction network.

Training objective. Minimize the average next-token cross-entropy loss. This loss measures how little probability the model assigns to the actual next token: lower loss means better prediction.

When using PyTorch's CrossEntropyLoss, pass:

- The network's raw output scores, called **logits**.
- The integer output index corresponding to each correct next token.

Do not apply softmax first; this loss function handles the necessary calculation internally.

Choosing the saved model. After each epoch, measure average loss on the clean validation records without updating any parameters. Save the model version with the lowest validation loss. If two epochs have exactly the same lowest loss, keep the earlier one. Use that saved version for subsequent clean/noisy evaluation.

Reproducibility. Record the software versions, whether training used a CPU or GPU, and settings used to make repeated runs reproducible. A fixed seed alone does not guarantee identical results across different devices. If hardware limitations require changes to the training settings, agree on and document those changes before comparing results, and apply them consistently to both models.

RESEARCH PROJECT • ASSIGNMENT 3

Evaluating the trained models on clean and noisy text

Train the mean-pooling model and the concatenation model once each, following Section 12. Select one saved version of each model using clean-validation loss, then keep its parameters unchanged throughout evaluation.

The three corruption seeds produce three noisy versions of the evaluation text. They do **not** require retraining either model.

13. Metrics and aggregation

For each model and evaluation condition, calculate **average loss, perplexity, and next-token accuracy**.

Average loss and perplexity

At each prediction position, record the probability that the model assigns to the actual observed next token.

$$S = -\Sigma_i \ln P(w_i \mid c_i)\,; \quad H = S / T\,; \quad PP = exp(H)$$

w_i is observed target i; c_i is its two-token context; P is the model's probability; ln is the natural logarithm. S is summed negative log loss; T counts scored targets including EOS; H is mean loss in nats per scored token; PP is perplexity. Sum loss and target counts over records first. Do not average record-level perplexities.

$$Accuracy = number\ of\ correct\ top-1\ predictions\ /\ T$$

To obtain a result for the entire evaluation condition, **add the losses and target counts across all retained records first**. Divide the total loss by the total target count, then calculate perplexity.

Do not average the individual records' perplexities.

14. Required result counts

An **evaluation condition** is either:

- The unchanged clean validation text; or
- A noisy version identified by its corruption operation, noise rate, and corruption seed.

Use the required operations—**insertion, deletion, and substitution**—at nonzero rates of **1%, 5%, 10%, and 20%**, with corruption seeds **11, 23, and 47**.

Result family	Expected rows
Per neural model	1 clean + (3 operations × 4 nonzero rates × 3 seeds) = 37 conditions.
Both neural models	74 condition rows. Each contains H, PP, accuracy and diagnostics
Noise summaries	24 rows: 2 models × 3 operations × 4 rates. For each metric report mean, minimum and maximum over the three noise seeds; show clean results separately.
Static span audit	37 condition rows shared across model variants. Save paired span-level evidence and validity counts.
N-gram control	37 additional rows for the Assignment 2 WordPiece interpolated baseline. Keep these separate from the 74 neural rows.

Interpreting variation across seeds

Use clean validation to select each neural model's saved version before evaluating the noisy conditions. Do not use noisy results to choose a different model or training setup.

The three corruption seeds show how results vary across different randomly generated corruptions. They do not measure variation across independently trained neural models, and their minimum-to-maximum range is **not a confidence interval**.

Label all results from this assignment **preliminary validation results**. Keep the reserved test data for the semester's final evaluation.

RESEARCH PROJECT • ASSIGNMENT 3

Controls and scientific interpretation

A useful result explains what changed, what stayed fixed and what the evidence cannot establish.

15. Compare with the Assignment 2 n-gram baseline

Compare both neural models with the Assignment 2 **WordPiece unigram/bigram/trigram mixture**, using identical validation records, tokenization, allowed output tokens, and scoring rules.

Reuse the existing n-gram counts if training records are unchanged; otherwise, rebuild them on the neural models' training subset. Reuse mixture weights only if the validation selection is unchanged; otherwise, reselect them using the Assignment 2 procedure. Save any rerun separately.

Compare clean performance and changes under each noise operation and rate.

Because the neural models use externally pretrained vectors and have different parameter counts, any advantage cannot be attributed to neural architecture alone.

16. Required figures and diagnosis

- Geometry: cosine similarity and raw Euclidean distance versus p, separated by operation. Show seed variation and valid-pair coverage; a decay or expansion is an observed trend, not a required curve shape.
- Prediction: H or PP versus p for mean, concatenation and the matched n-gram control, with clean baselines. Also show next-token accuracy; retain all three metrics in the results file.
- Tokenization: relate results to unknown-token rate, pieces per original clean span/record and actual edit distance. Define each denominator; changing token counts can change the prediction task.
- Trace at least three clean/noisy examples with IDs, text, token pieces, geometric measurements and model outcomes. Explain at least one counterexample to a simple "small drift means robust prediction" story, if present.

17. Limits on conclusions

A prediction can change when the highest-scoring class changes, but this study does not prove a fixed geometric decision-boundary threshold at p = 0.20. Noise may remove distinctions, map inputs to [UNK], or be erased by tokenizer normalization. Inspect these mechanisms before claiming semantic tolerance.

RESEARCH PROJECT • ASSIGNMENT 3

Verification and team responsibilities

One shared experiment and paper; two implementation leads with mutual review.

18. Tests to pass before the full grid

Check	Required evidence
Data continuity	IDs and text hashes match the retained Assignment 2 artifacts; no split crossing, resampling or independent noisy truncation.
Alignment	A substitution, an insertion, fragmentation into more subwords and a fully deleted span keep correct original span IDs.
Geometry	Identical nonzero vectors yield cosine 1 and distance 0; orthogonal unit vectors yield cosine 0 and distance √2. Undefined spans are counted.
Causal targets	Hand-trace a two-token record: three targets including EOS. Verify BOS padding, no future-token access and record resets.
Network and freeze	Input/hidden/output shapes match Section 11; E checksum unchanged after training; probabilities sum to 1 within tolerance.
Loss aggregation	Hand-compute a tiny example and match code. Pool summed losses and counts, then exponentiate once.
Evaluation state	No parameter updates during evaluation. One frozen checkpoint per variant; clean condition scored once; empty noisy records still score EOS.

19. Division of work

- Student A leads aligned span selection, embedding extraction, geometric diagnostics and noise-manifest integrity. Student B reviews the alignment and metric tests.
- Student B leads context/target construction, the two neural variants, training and predictive metrics. Student A checks target shifting, frozen parameters and loss aggregation.
- Both rerun the shared tests, inspect the paired examples, maintain the baseline comparison, create figures from saved results and write the cumulative paper. Both must explain the complete pipeline.

RESEARCH PROJECT • ASSIGNMENT 3

Submission and progress toward the paper

Submit one team package.

20. What to submit

- Reproducible code: noise_vector_drift.py, neural_next_token.py (or documented legacy filename), tests, environment file and a README with exact commands. Include the two frozen neural checkpoints or a documented accessible course-approved location.
- Experiment evidence: configuration, selected IDs/hashes, checkpoint/tokenizer revision, output mapping, corruption manifests or references to unchanged artifacts, per-record predictive results, span-level diagnostics, condition summaries and training/checkpoint logs. Do not redistribute the corpus if its license does not permit it.
- Cumulative paper v0.3: ACL-style PDF plus editable LaTeX source. Target up to six main-text pages; references and compact reproducibility appendices are additional. Use the official ACL template. Include both students' contributions and explicit preliminary-validation labeling.

Paper component	Status after Assignment 3
Introduction / related work	Refined research questions and motivation; add static embeddings and neural language-model sources.
Data / experimental methods	Extend the existing corpus/noise protocol with alignment, frozen vectors, model equations, training recipe and controls.
Results / analysis	Retain the classical findings; add geometric and neural validation evidence, error examples and uncertainty limitations.
Discussion / ethics	Explain possible mechanisms, contradictory findings, external pretraining and what metrics cannot establish.
Still ahead	Contextual transformer experiments and any later approved extensions; final frozen test evaluation and final synthesis. No claim of completed BERT evaluation yet.

References: Jurafsky & Martin, supplied third-edition draft, Chapters 5–6; Bengio et al. (2003), A Neural Probabilistic Language Model, https://jmlr.org/papers/v3/bengio03a.html; Devlin et al. (2019), BERT, https://aclanthology.org/N19-1423/; checkpoint: https://huggingface.co/google-bert/bert-base-uncased; Kaplan et al. (2020), https://arxiv.org/abs/2001.08361; ACL template: https://github.com/acl-org/acl-style-files. Consult your pinned PyTorch version's CrossEntropyLoss documentation for implementation details.